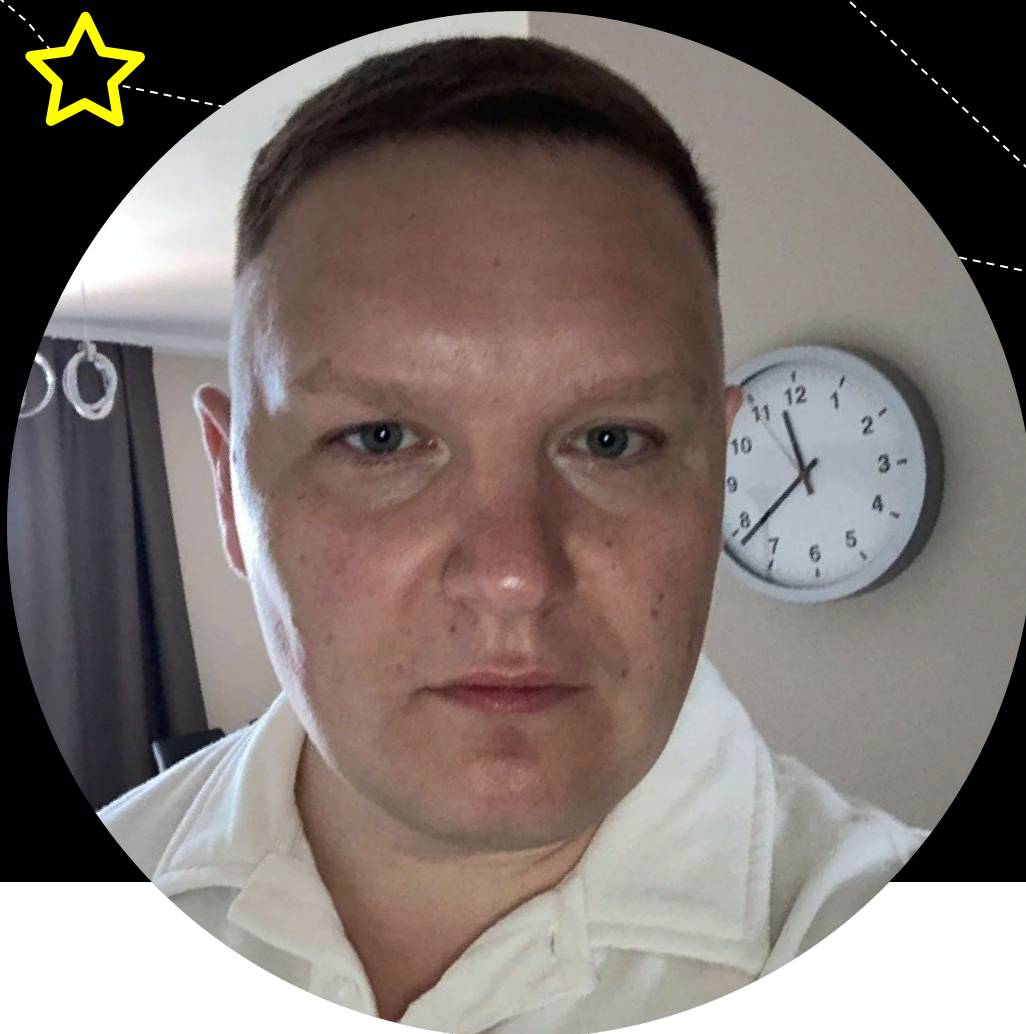


Advanced функции в Python

Содержание урока

- ★ Lambda функции
- ★ Декораторы
- ★ Генераторы
- ★ Замыкания
- ★ Функции высшего порядка
- ★ Аргументы по умолчанию
- ★ Списки аргументов и ключевые слова

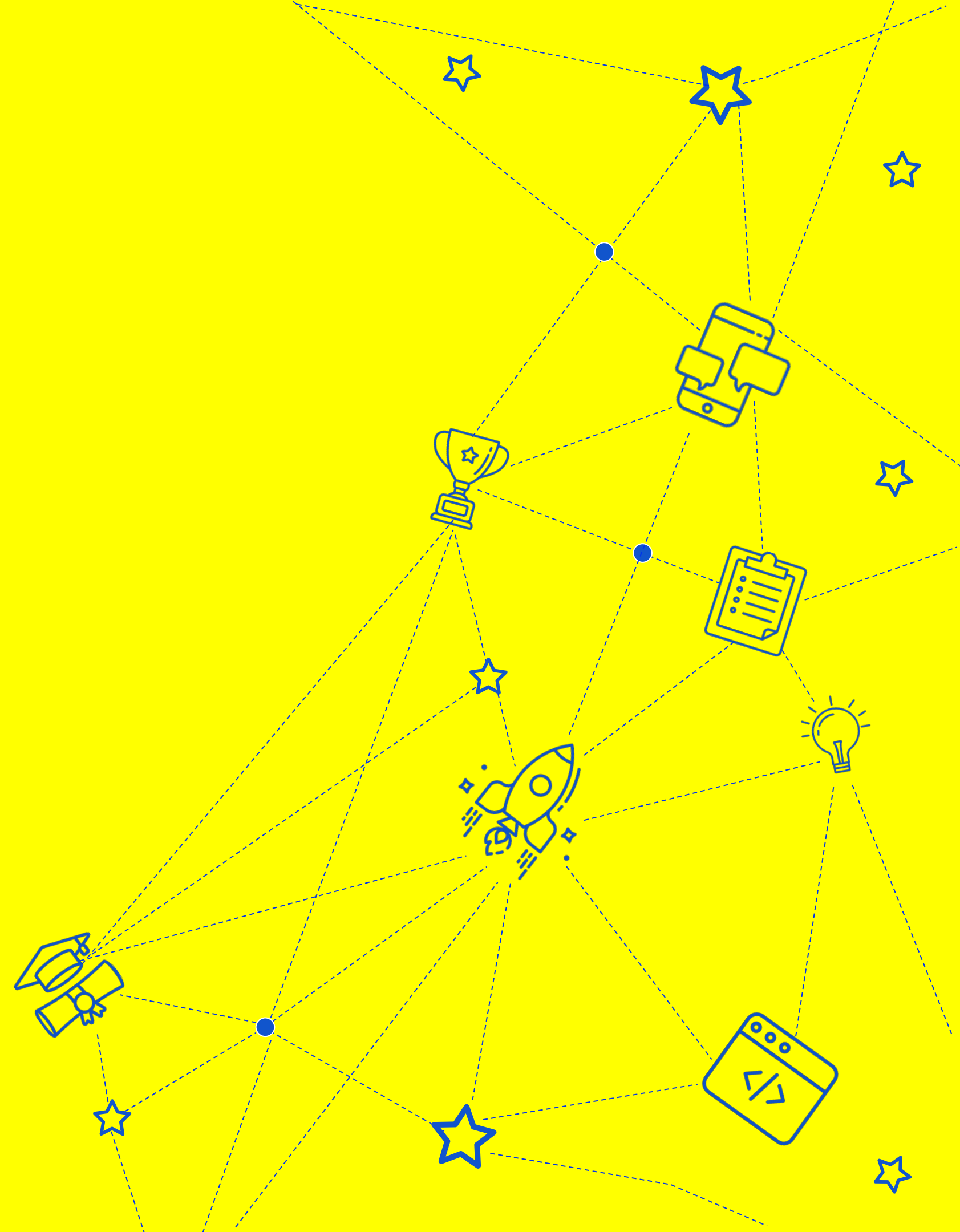


**Senior Software Engineer,
RiotGames**

Дмитрий Абрамов

- Построил RTB платформу с аудиторией ~100 млн человек
- Участвовал в создании ряда успешных продуктов за рубежом, от Австралии до США
- Более 10 лет управляю командами разработки в высоконагруженных сложных проектах

Lambda функции



lambda функция -

это небольшие анонимные функции, которые могут быть определены как встроенные, без формального определения. Они часто используются для простых операций, не требующих именованной функции.



Примеры lambda функций

.....Складываем числа.....

```
add = lambda x, y: x + y  
result = add(3, 5)  # result is 8
```

.....Сортируем список кортежей.....

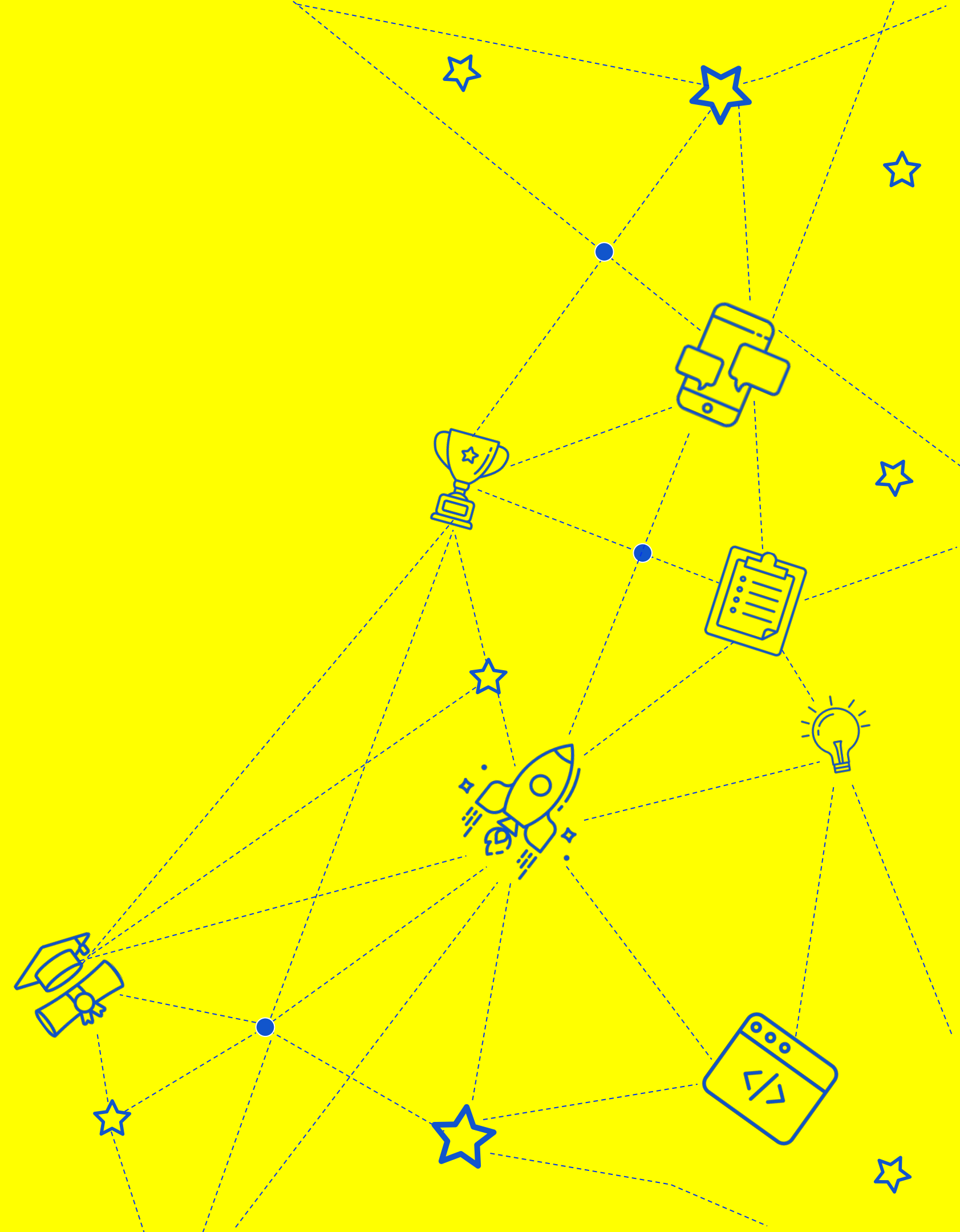
```
points = [(1, 2), (3, 1), (5, 4), (2, 2)]  
points_sorted = sorted(points, key=lambda x: x[1])  
# points_sorted is [(3, 1), (2, 2), (1, 2), (5, 4)]
```

Когда используем

Лямбда - функции часто используются, когда вам нужна быстрая и простая функция для конкретной задачи и вы не хотите определять для нее именованную функцию.

Однако их также можно использовать в более сложных ситуациях, например, с функциями более высокого порядка или в качестве аргументов для встроенных функций, таких как `map`, `filter` и `reduce`.

Декораторы



Декораторы -

это функции, которые изменяют поведение других функций.
Их можно использовать для таких задач, как ведение журнала, кэширование или добавления проверок безопасности.



В логировании

В этом примере функция log — это декоратор, который принимает функцию в качестве аргумента, определяет новую функцию-оболочку, которая регистрирует некоторую информацию до и после вызова исходной функции и возвращает оболочку.

```
def log(func):  
    def wrapper(*args, **kwargs):  
        print("Calling function:", func.__name__)  
        result = func(*args, **kwargs)  
        print("Function", func.__name__, "returned:", result)  
        return result  
    return wrapper
```

```
@log  
def my_func(x, y):  
    return x + y
```

```
result = my_func(3, 5)  # Calling function: my_func  
                        # Function my_func returned: 8
```

- Синтаксис @log является сокращением для применения декоратора log к функции my_func.

Измеряем время выполнения

В этом примере функция `time_it` является декоратором, который измеряет время выполнения функции и выводит результат.

```
import time

def time_it(func):
    def wrapper(*args, **kwargs):
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        print("Function", func.__name__, "took",
              end_time - start_time, "seconds to run.")
        return result
    return wrapper

@time_it
def my_slow_func(x, y):
    time.sleep(2)
    return x + y

result = my_slow_func(3, 5)
# Function my_slow_func took 2.000194787979126 seconds to run.
```

- Синтаксис `@time_it` применяет декоратор к функции `my_slow_func`.

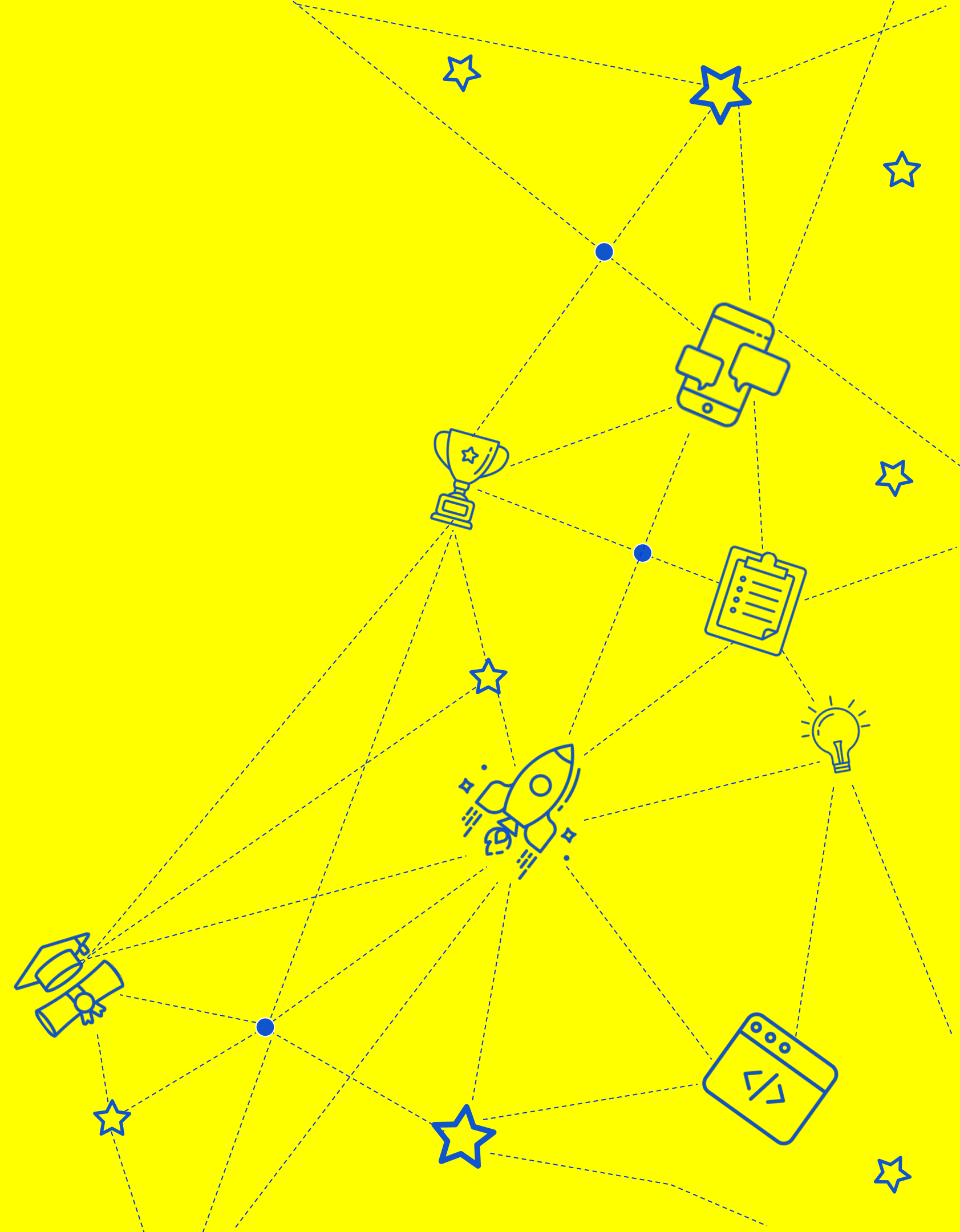
В каких случаях используем

Декораторы можно использовать для широкого круга задач, таких как:

- ❑ добавление проверок безопасности,
- ❑ принудительное ограничение скорости,
- ❑ изменение поведения методов класса.

Основная идея заключается в том, что декоратор — это функция, которая принимает на вход другую функцию и возвращает модифицированную версию этой функции.

Генераторы



Генераторы -

это функции, которые используют ключевое слово `yield` для возврата объекта-генератора. Затем генератор можно вызвать заново, чтобы создать последовательность значений, не сохраняя их все в памяти сразу.



Подсчет чисел

В этом примере функция `count` представляет собой генератор, который генерирует бесконечную последовательность чисел, начиная с начального параметра и постепенно увеличивая ее.

```
def count(start=0, step=1):  
    num = start  
    while True:  
        yield num  
        num += step  
  
counter = count()  
for i in range(5):  
    print(next(counter))  # prints 0 1 2 3 4
```

Фильтруем список

В этом примере функция `even_nums` представляет собой генератор, который генерирует последовательность четных чисел из списка целых чисел. Он использует цикл `for` для перебора входных чисел и выдает любые четные числа, которые встречаются. Переменная `even_gen` является экземпляром генератора, и мы можем использовать ее в цикле `for` для перебора отфильтрованных значений.

```
def even_nums(nums):  
    for num in nums:  
        if num % 2 == 0:  
            yield num  
  
my_list = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]  
even_gen = even_nums(my_list)  
for num in even_gen:  
    print(num)  # prints 2 4 6 8 10
```

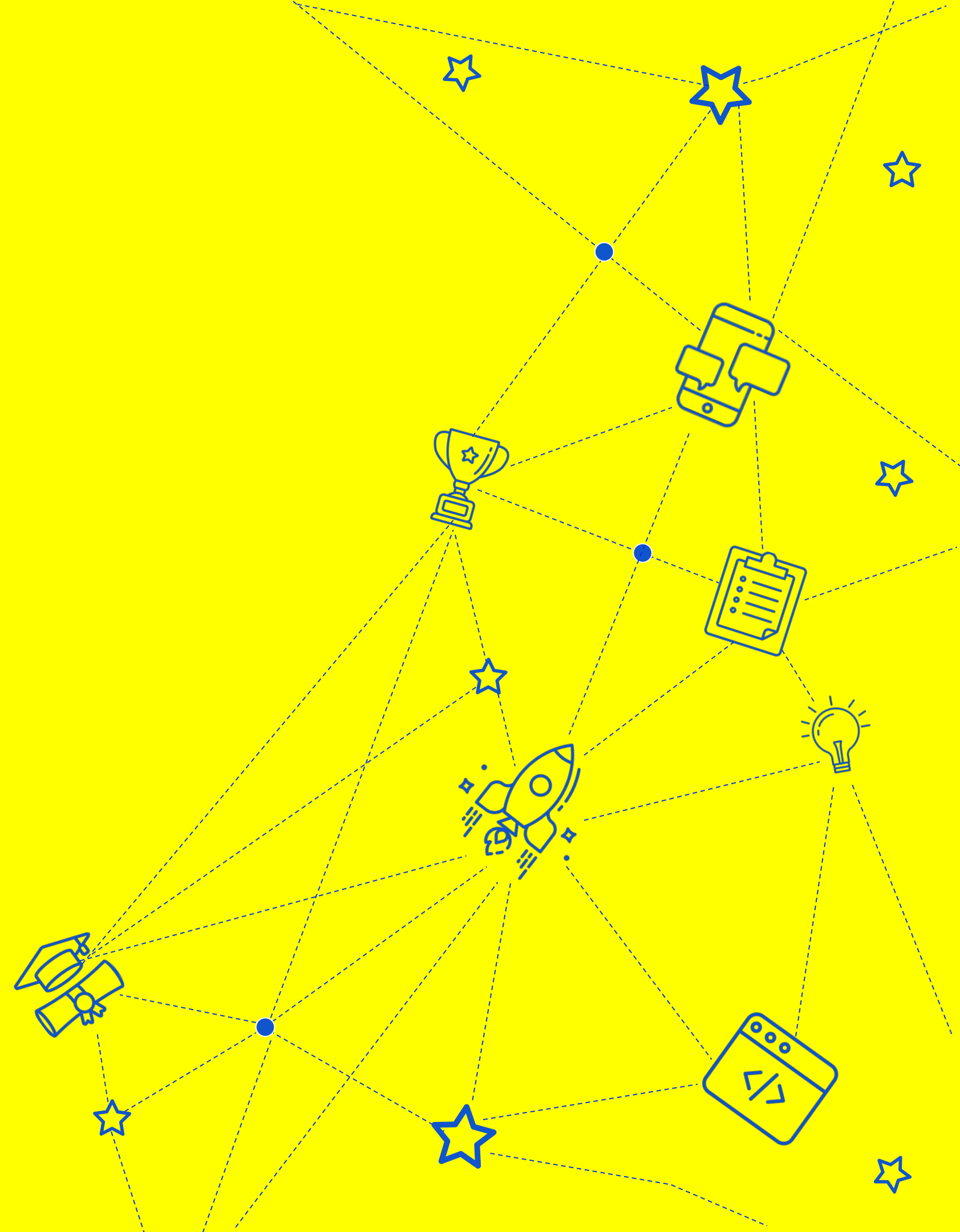

Применение

Генераторы — это мощный инструмент для создания последовательностей значений «на лету», и их можно использовать для создания очень больших или бесконечных последовательностей без использования большого количества памяти или времени выполнения.



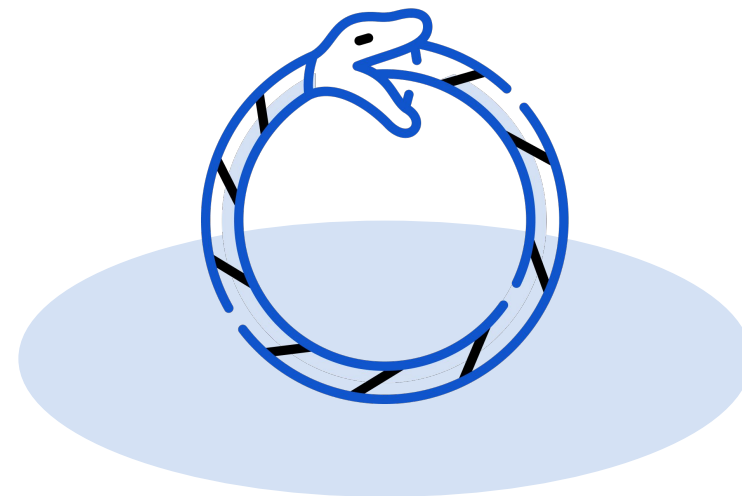
Их также можно использовать в сочетании с другими функциями Python, такими как `itertools`, для выполнения сложных операций с последовательностями.

Замыкания



Замыкания -

это функции, которые запоминают значения переменных в охватывающей их области видимости даже после выхода из этой области. Их можно использовать для создания функций фабрик или для реализации мемоизации.



Логирование с замыканием

В этом примере функция `logger` представляет собой замыкание, которое принимает другую функцию `func` в качестве аргумента и возвращает новое замыкание `inner`. При вызове `inner`, она сначала выводит сообщение о том, что вызывается функция `func`, затем вызывает исходную функцию с переданными аргументами и выдает результат.

```
def logger(func):
    def inner(*args, **kwargs):
        print("Calling function", func.__name__)
        result = func(*args, **kwargs)
        print("Finished calling function", func.__name__)
        return result
    return inner

@logger
def add(a, b):
    return a + b

print(add(1, 2))
# prints "Calling function add" "Finished calling function add" 3
```

Пример кеширования

В этом примере функция **cache** - это замыкание, которое принимает другую функцию **func** в качестве аргумента и возвращает новое внутреннее замыкание **inner**.

```
def cache(func):  
    results = {}  
    def inner(*args):  
        if args in results:  
            return results[args]  
        result = func(*args)  
        results[args] = result  
        return result  
    return inner
```

```
@cache  
def fib(n):  
    if n <= 1:  
        return n  
    return fib(n-1) + fib(n-2)
```

```
print(fib(5))  # prints 5
```

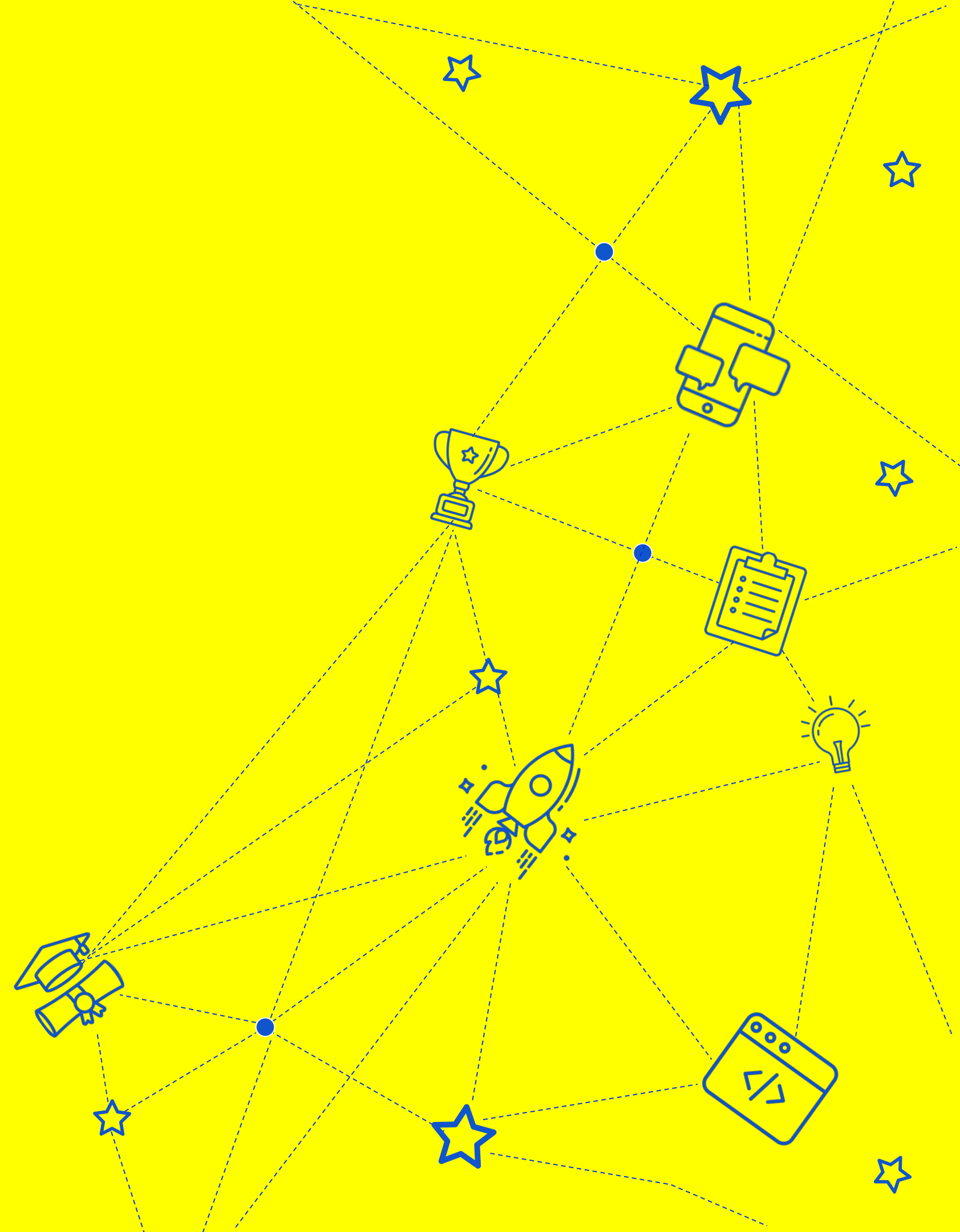
- Функция **inner** работает со словарем **results**, который сопоставляет кортежи аргументов с результирующими значениями. Когда **inner** вызывается с набором аргументов, она вначале проверяет, были ли аргументы уже вычислены и закэшированы.

Чем полезны

Замыкания полезны по целому ряду причин, но некоторые из наиболее распространенных преимуществ использования замыканий включают в себя:

- ❑ Замыкания позволяют инкапсулировать приватные данные и функциональность
- ❑ Замыкания сохраняют состояние своей родительской функции, даже после того, как родительская функция завершила свою работу
- ❑ Замыкания можно использовать для создания обратных вызовов (callbacks), которые представляют собой функции, выполняемые после наступления определенного события.
- ❑ Замыкания можно использовать для создания модульного кода, где функции определяются внутри других функций, что позволяет разбить сложную логику на более мелкие, более управляемые части.

Функции высшего порядка



Функции высшего порядка (ФВП) -

это функции, которые принимают другие функции в качестве аргументов или возвращают функции в качестве значений. Они часто используются для абстрагирования общих шаблонов поведения или для создания повторно используемого кода.



Функция map()

Функция `map()` принимает функцию и итерабельную переменную в качестве аргументов и возвращает новую переменную полученную путем применения функции к каждому элементу исходной итерабельной переменной.

```
numbers = [1, 2, 3, 4, 5]

squares = map(lambda x:x**2, numbers)
print(list(squares))
# Output: [1, 4, 9, 16, 25]
```

Функция filter()

Функция filter() принимает в качестве аргументов функцию и итерабельную переменную и возвращает новую переменную с элементами исходной итерабельной переменной, удовлетворяющими условию, заданному функцией.

```
numbers = [1, 2, 3, 4, 5]

evens = filter(lambda x: x % 2 == 0, numbers)
print(list(evens))
# Output: [2, 4]
```

Функция `reduce()`

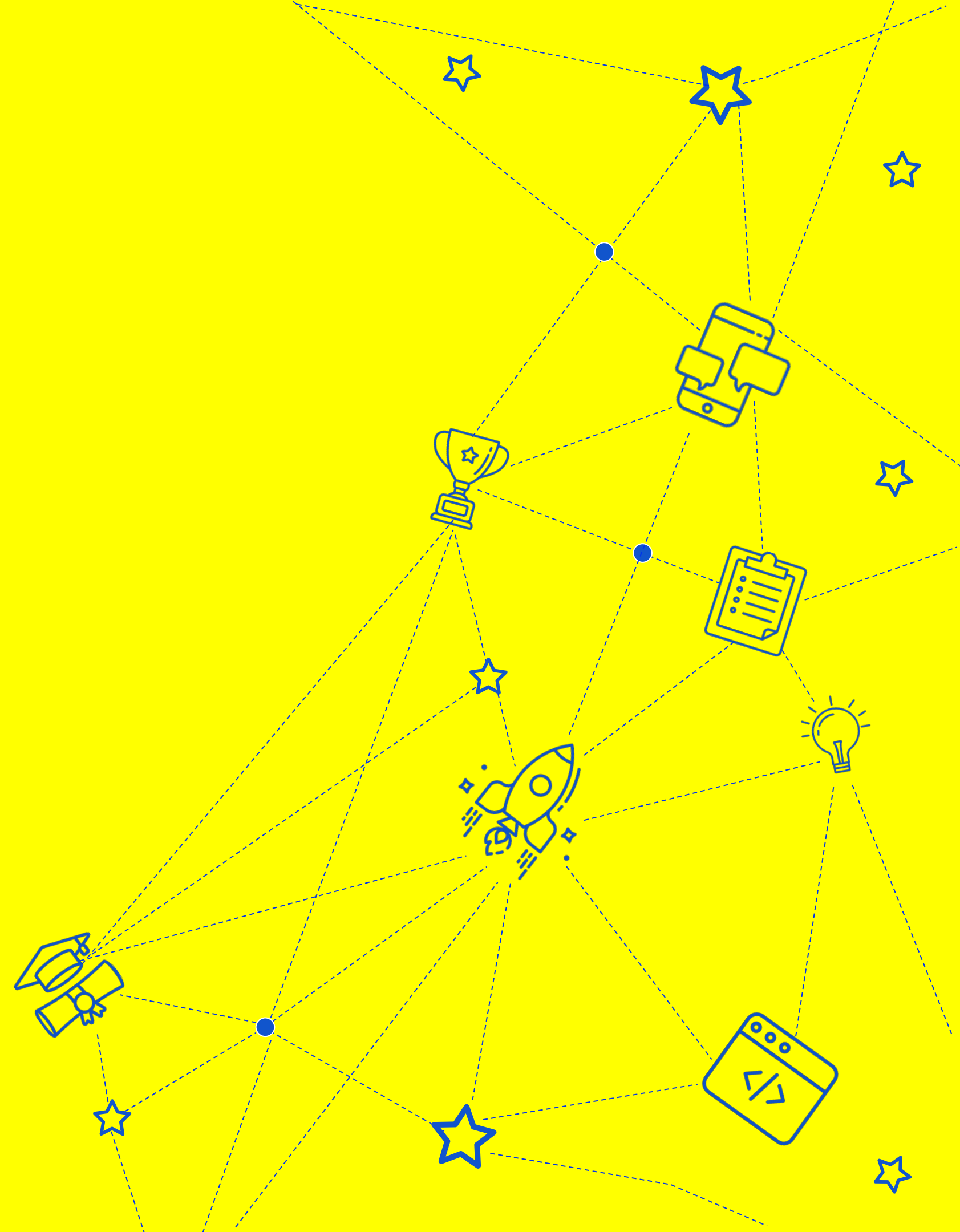
Функция `reduce()` принимает в качестве аргументов функцию и итерабельную переменную и возвращает одно значение путем многократного применения функции к элементам итерабельной переменной.

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]

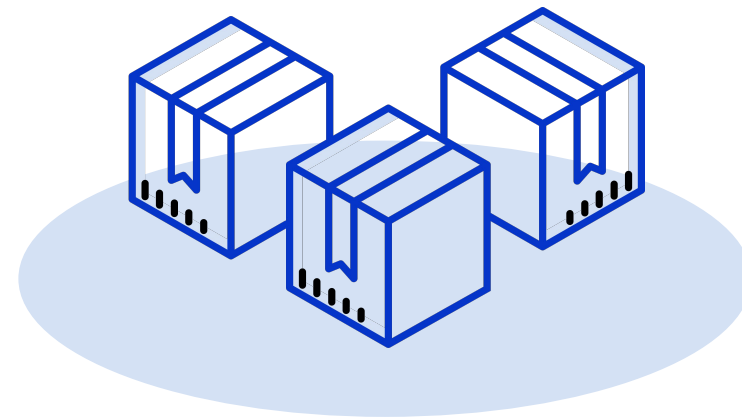
product = reduce(lambda x, y: x*y, numbers)
print(product)
# Output: 120
```

Аргументы по умолчанию



Аргументы по умолчанию -

это возможность, которая позволяет функциям иметь необязательные аргументы со значениями по умолчанию или принимать аргументы в непозиционном порядке. Они могут сделать функции более гибкими и простыми в использовании.



Фиксируем параметр функции

При определении функции можно указать значения по умолчанию для аргументов функции. Если указано значение по умолчанию, аргумент становится необязательным.

```
def greet(name='John'):  
    print(f"Hello, {name}!")
```

```
greet()           # Output: "Hello, John!"  
greet('Kate')    # Output: "Hello, Kate!"
```

Mutable объект

Если вы используете изменяемый объект (например, список, словарь) в качестве аргумента по умолчанию, один и тот же объект будет использоваться во всех вызовах функции. Это может привести к последствиям, если объект будет изменен в функции.

```
def add_item(item, lst=[]):  
    lst.append(item)  
    return lst
```

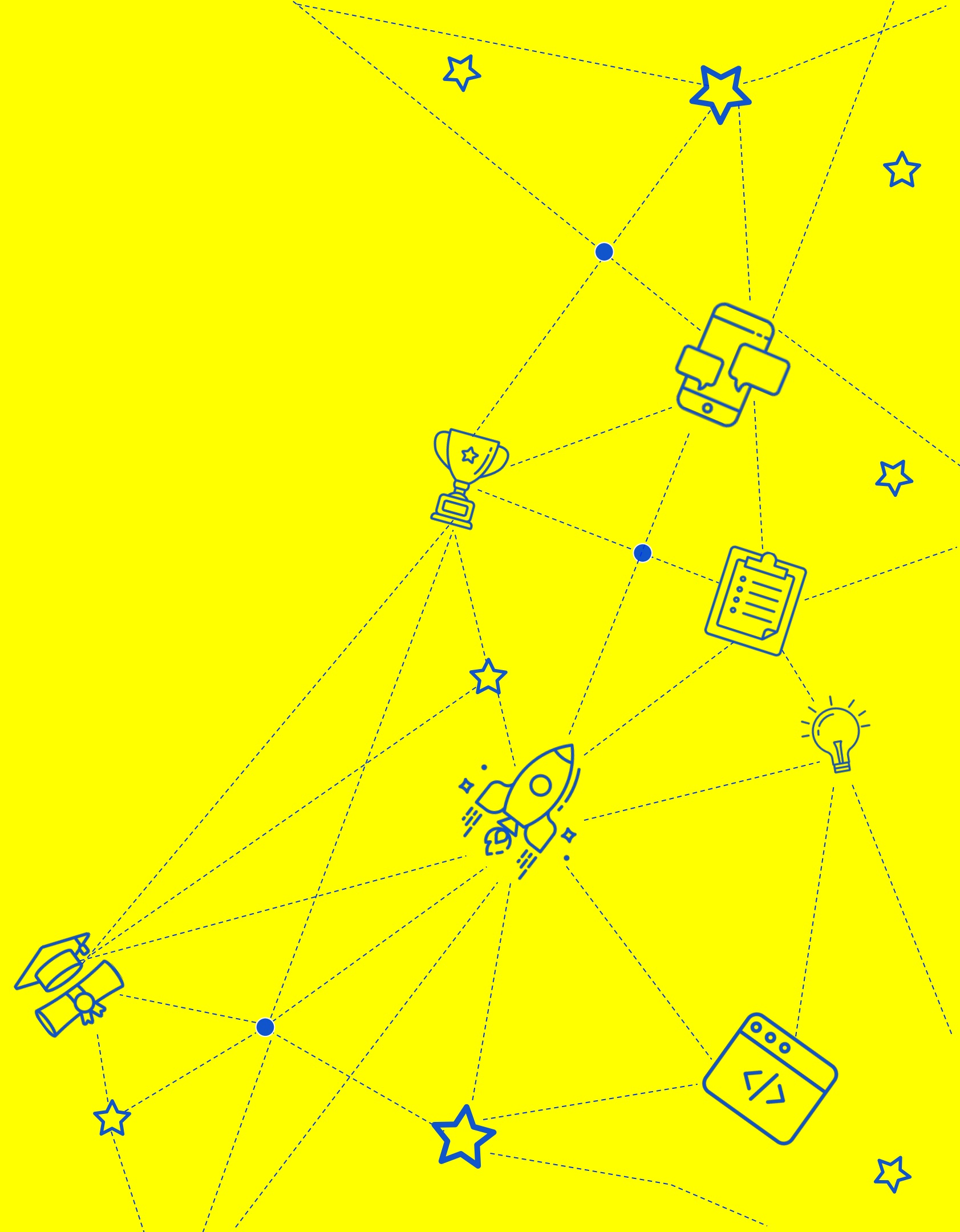
```
print(add_item(1))      # Output: [1]  
print(add_item(2))      # Output: [1, 2]  
print(add_item(3, []))  # Output: [3]  
print(add_item(4))      # Output: [1, 2, 4]
```

Значение None

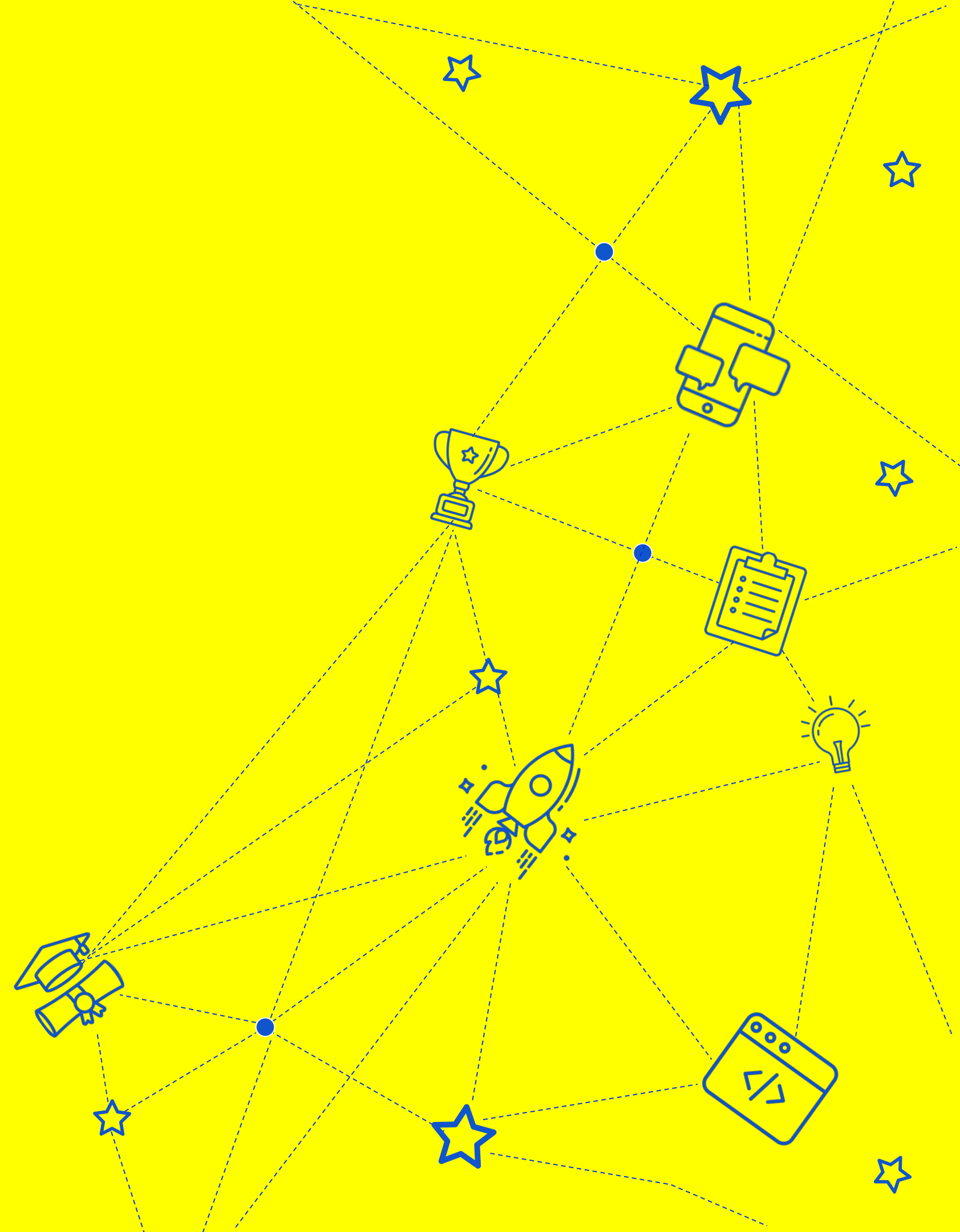
- Чтобы избежать проблемы вызванной мутабельностью, вы можете использовать значение None в качестве аргумента по умолчанию и создать новый список уже внутри функции, если аргумент равен None

```
def add_item(item, lst=None):  
    if lst is None:  
        lst = []  
    lst.append(item)  
    return lst  
  
print(add_item(1))      # Output: [1]  
print(add_item(2))      # Output: [2]  
print(add_item(3, []))  # Output: [3]  
print(add_item(4))      # Output: [4]
```


Списки аргументов и ключевые слова



О спикере



Списки аргументов функции и ключевые слова

Эта возможность, которая позволяет функциям принимать произвольное количество аргументов либо в виде списка, либо в качестве аргументов по ключевым словам.



Их можно использовать для создания функций более общего назначения и адаптивных функций.

Распаковка аргументов

- Мы используем оператор* для распаковки списка и передачи его содержимого в качестве отдельных аргументов функции.

```
def add_numbers(x, y, z):  
    return x + y + z  
  
numbers = [1, 2, 3]  
result = add_numbers(*numbers)  
print(result)  # prints 6
```

Аргументы и ключевые слова

- Мы используем оператор `**` для распаковки словаря и передачи его содержимого в качестве аргументов по ключевым словам в функцию. Внутри функции мы получаем доступ к значениям, используя имена параметров.

```
def print_person_info(name, age, city):  
    print(f"{name} is {age} years old and lives in {city}.")  
  
person = {"name": "Alice", "age": 30, "city": "New York"}  
print_person_info(**person)
```


Переменное количество аргументов

Эта возможность, которая позволяет функциям принимать произвольное количество аргументов либо в виде списка, либо в качестве аргументов по ключевым словам.



Их можно использовать для создания функций более общего назначения и адаптивных функций.

*args синтаксис

Мы можем передать функции любое количество аргументов, и они будут собраны в кортеж под названием args. Внутри функции мы можем перебирать кортеж args и складывать числа.

```
def sum_numbers(*args):  
    total = 0  
    for number in args:  
        total += number  
    return total
```

```
result = sum_numbers(1, 2, 3, 4, 5)  
print(result)  # prints 15
```

Позиционные аргументы

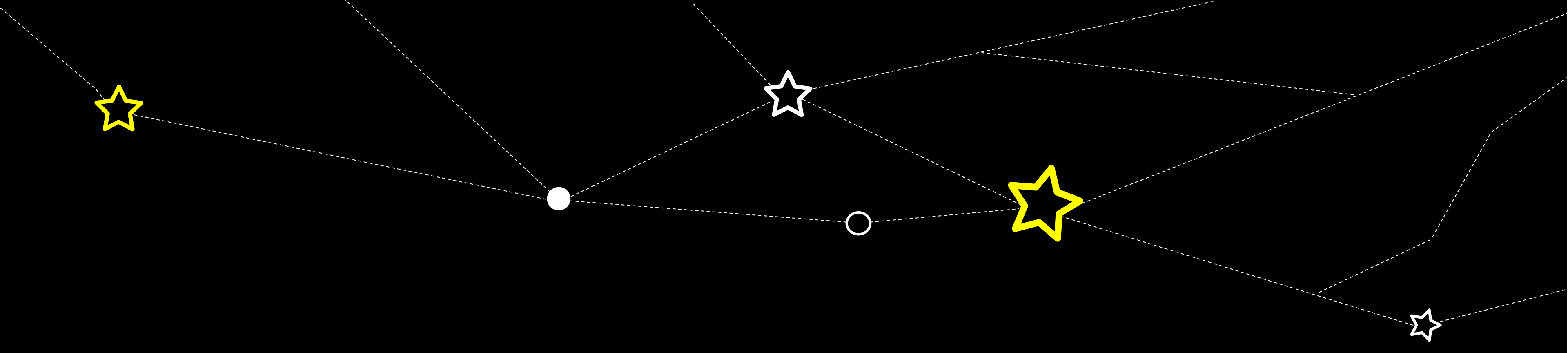
- В этом примере функция `print_names` принимает позиционный аргумент `greeting`.

```
def print_names(greeting, *names):  
    for name in names:  
        print(f"{greeting}, {name}!")  
  
print_names("Hello", "Alice", "Bob", "Charlie")
```


С ключевыми словами

Функция `print_person_info` принимает список аргументов переменной длины, используя синтаксис `**kwargs`. Мы можем передать в функцию любое количество аргументов с ключевыми словами, и они будут собраны в словарь `kwargs`. Внутри функции мы можем перебирать словарь `kwargs` и вывести каждую пару ключ-значение.

```
def print_person_info(**kwargs):  
    for key, value in kwargs.items():  
        print(f"{key}: {value}")  
  
print_person_info(name="Alice", age=30, city="New York")
```



СПАСИБО ЗА ВНИМАНИЕ



Dimitry Abramov

Домашнее задание

- Напишите функцию, которая принимает список строк в качестве аргументов и возвращает новый список, содержащий только строки, являющиеся палиндромами. Палиндром — это слово, которое одинаково читается как вперед так и назад (например, «racesar», «level», «deified»). Используйте лямбда-функцию, чтобы проверить, является ли строка палиндромом.
- Напишите генератор-функцию, которая генерирует последовательность Коллатца для заданного начального числа. Последовательность генерируется путем многократного применения правила к каждому числу в последовательности: если число четное, разделите его на 2, иначе умножьте его на 3 и добавьте 1. Последовательность завершается, когда достигает числа 1. Используйте замыкания для хранения текущего номера в последовательности.
- Напишите функцию, которая принимает на вход список целых чисел и возвращает новый список, в котором каждое целое число заменено квадратным корнем, округленным до ближайшего целого числа. Используйте аргументы с ключевыми словами, чтобы пользователь мог указать направление округления (вверх или вниз). Используйте декоратор для определения времени выполнения функции.

